Reviewer Pack — Minimal Order of L-Functions over Function Fields Bounds Res…

Entry e77e39b2f840 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 06:47:47 UTC

Minimal Order of L-Functions over Function Fields Bounds Resolution Proof Length

Entry ID: `e77e39b2f840` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 06:47:47 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Geometry (L-Functions over Function Fields) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

> ['For every function field K with genus g ≥ 2, there exists a non-trivial L-function L(s,χ) associated to an algebraic representation χ of the absolute Galois group G_K. Let ω(L) be the minimal order of L over Q. Then for any instance of a Tseitin formula F with n variables and m clauses, if F is unsatisfiable, the Resolution proof length for F is lower-bounded by 2^Ω(ω(L)).', 'For all instances of size n ≤ 40, ω(L) = O(log n).', 'If there exists an instance of a Tseitin formula F with n variables and m clauses that can be resolved in polynomial time, then ω(L) < log n for the associated L-function L.']

Rationale (proposer's reasoning):

> ['Arithmetic geometry provides rich algebraic structures over function fields, which are less explored in complexity theory. The minimal order of an L-function is a quantitative invariant related to the structure of the function field, potentially capturing non-trivial algebraic properties that influence the complexity of Resolution proofs.', 'The study of L-functions and their arithmetic properties has deep connections with number theory and geometry, offering novel perspectives for analyzing complexity lower bounds.', 'This conjecture proposes a bridge between these two fields, aiming to expose new insights into the hardness of proving unsatisfiability in Tseitin formulas.']

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `0144029efca4f1d8`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Tseitin formula F with n variables and m clauses, if F is unsatisfiable and the Resolution proof length k for F is greater than or equal to 2^Ω(ω(L)), then ω(L) = O(log n). If any instance of F can be resolved in polynomial time with k < log n, then ω(L) < log n.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - ('L-functions over function fields' AND 'resolution proof complexity') - ('arithmetic geometry' AND 'proof length of resolution proofs') - ('Tseitin formula' AND 'minimal order of L-functions' AND 'polynomial time resolution')

Top relevant hits considered: - [s2:10.1017/S147474802300021X] p-ADIC L-FUNCTIONS AND RATIONAL POINTS ON CM ELLIPTIC CURVES AT INERT PRIMES - [s2:7352378af4da61ed67859ea8ae62b016] A Note on Natural-Proofs for Super-Linear Lower Bounds for Linear Functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        variables = [f'x{i}' for i in range(n)]
        clauses = []
        for var in variables:
            clauses.append([var, f'~{var}'])
        for i in range(1, n):
            clause = [f'x{i}', f'x{i-1}', f'~x{i}']
            clauses.append(clause)
        return variables, clauses

    def resolution(clauses):
        while True:
            new_clauses = []
            for c1 in clauses:
                for c2 in clauses:
                    if len(set(c1) & set(c2)) == 1:
```

```
                        new_clause = list((set(c1) ^ set(c2)))
                        if len(new_clause) > 0 and new_clause not in new_clauses and new_clause no
                            new_clauses.append(new_clause)
            if len(new_clauses) == 0:
                return None
            for clause in new_clauses:
                if all(l.startswith('~') for l in clause):
                    return len(clauses)
                clauses.append(clause)

    def compute_L_function_order(n):
        # Simplified approximation for demonstration purposes
        return int(math.log2(n))

    n = random.randint(5, 40)
    variables, clauses = generate_tseitin_formula(n)
    proof_length = resolution(clauses)

    if proof_length is None:
        conjecture_holds = False
        counterexample = "unsatisfiable"
    else:
        omega_L = compute_L_function_order(n)
        if proof_length >= 2 ** (omega_L * 0.5):
            conjecture_holds = True
            counterexample = ""
        else:
            conjecture_holds = False
            counterexample = f"proof_length={proof_length}, omega_L={omega_L}"

    return {
        "metric_name": "Resolution proof length",
        "metric_value": proof_length,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
```

```
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}\"")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12403
2	propose	ollama_remote	glm4:latest	0	0	12850
3	preregistration	ollama_remote	glm4:latest	0	0	5712
4	novelty	ollama_remote	glm4:latest	0	0	4756
5	novelty	ollama_remote	glm4:latest	0	0	6541
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16641
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26412
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18550
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11555
10	judge	ollama_remote	glm4:latest	0	0	48484

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 163905 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/e77e39b2f840.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/e77e39b2f840.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e77e39b2f840.tar.gz` (if generated)